Задана квадратна матриця невідомого розміру. Отримати матрицю, в якій крайні елементи побічної діагоналі поміняються місцями з сусідніми по діагоналі елементами.

У цій програмі матриця задається як список списків. Кожен внутрішній список є окремим рядком матриці. Програма бере квадратну матрицю, проходить по всіх її рядках і по всіх елементах кожного рядка, після чого формує нову матрицю. Під час формування нової матриці перевіряється, чи поточний елемент належить до крайніх елементів побічної діагоналі або до сусідніх із ними елементів на цій діагоналі. Якщо так, то ці елементи міняються місцями. Усі інші елементи залишаються без змін.

1. Оголошення модуля і експорт функцій

```
-module(lab6_var16).
-export([matrix_main/1, matrix_print/1, matrix_data/1, matrix_pro/5,
row_pro/6]).
```

Тут задається назва модуля `lab6_var16`. Далі через `-export` перелічуються функції, які можна викликати ззовні. Основною функцією є `matrix_main/1`, вона запускає обробку матриці. Функція `matrix_print/1` використовується для зручного виводу результату по рядках. Інші функції є допоміжними і реалізують сам алгоритм.

---

2. Тестові матриці

```
matrix_data(1) ->
    [[1,2,3,4,5],
     [6,7,8,9,10],
     [11,12,13,14,15],
     [16,17,18,19,20],
     [21,22,23,24,25]];

matrix_data(2) ->
    [[1,2,3,4],
     [5,6,7,8],
     [9,10,11,12],
     [13,14,15,16]].
```

Функція `matrix_data/1` містить готові тестові матриці. Якщо передати значення `1`, програма візьме першу матрицю розміру `5 на 5`. Якщо передати `2`, буде взята друга матриця розміру `4 на 4`. Це потрібно для перевірки роботи алгоритму на різних прикладах.

---

3. Основна функція запуску

```
matrix_main(K) ->
    Ms = matrix_data(K),
    N = length(Ms),
    matrix_pro(Ms, 1, N, Ms, []).
```

Функція `matrix_main/1` є основною. Спочатку вона отримує тестову матрицю через `matrix_data(K)`. Потім визначає розмір матриці за допомогою `length(Ms)`. Після цього запускається функція `matrix_pro/5`, яка буде рекурсивно обробляти матрицю по рядках. Тут `1` означає, що обробка починається з першого рядка, а порожній список `[]` використовується як накопичувач для нової матриці.

---

4. Функція красивого виводу

```
matrix_print(K) ->
    M = matrix_main(K),
    print_rows(M).
```

Ця функція потрібна лише для зручного показу результату. Вона викликає `matrix_main/1`, отримує вже перетворену матрицю і передає її у функцію `print_rows/1`, яка друкує кожен рядок окремо. Це зручно на захисті, бо результат виглядає як справжня матриця.

---

5. Друк рядків матриці

```
print_rows([Row | Rs]) ->
    io:format("~w~n", [Row]),
    print_rows(Rs);
print_rows([]) ->
    ok.
```

Функція `print_rows/1` рекурсивно проходить по списку рядків. Якщо рядок є, він виводиться через `io:format`, після чого функція переходить до наступного рядка. Якщо рядки закінчилися, повертається `ok`. Таким чином матриця друкується по одному рядку.

---

6. Обробка матриці по рядках

```
matrix_pro([R | Rs], I, N, Ms, Acc) ->
    matrix_pro(Rs, I + 1, N, Ms, Acc ++ [row_pro(R, I, 1, N, Ms, [])]);
matrix_pro([], _, _, _, Acc) ->
    Acc.
```

Функція `matrix_pro/5` виконує рекурсивний обхід усіх рядків матриці.
`R` — поточний рядок.
`Rs` — решта рядків.
`I` — номер поточного рядка.
`N` — розмір матриці.

`Ms` — початкова матриця.
`Acc` — накопичувач нової матриці.

Для кожного рядка викликається функція `row_pro/6`, яка будує новий рядок. Потім цей рядок додається до нової матриці. Коли всі рядки оброблені, функція повертає готову матрицю.

---

7. Обробка одного рядка по елементах

Саме тут реалізована основна логіка перестановки.

Перша перестановка

```
row_pro([_X | Xs], I, J, N, Ms, Acc) when I =:= 1, J =:= N ->
    row_pro(Xs, I, J + 1, N, Ms, Acc ++ [get_elem(Ms, 2, N - 1)]);
```

Тут перевіряється, чи поточний елемент має координати `(1,N)`, тобто є верхнім крайнім елементом побічної діагоналі. Якщо так, то замість нього вставляється елемент з координатами `(2,N-1)`, тобто сусідній елемент по діагоналі.

```
row_pro([_X | Xs], I, J, N, Ms, Acc) when I =:= 2, J =:= N - 1 ->
    row_pro(Xs, I, J + 1, N, Ms, Acc ++ [get_elem(Ms, 1, N)]);
```

Ця гілка виконує зворотну заміну. Якщо поточний елемент стоїть у позиції `(2,N-1)`, то на його місце записується значення з позиції `(1,N)`. Так реалізується перестановка першої пари елементів.

---

8. Друга перестановка

```
row_pro([_X | Xs], I, J, N, Ms, Acc) when I =:= N - 1, J =:= 2 ->
    row_pro(Xs, I, J + 1, N, Ms, Acc ++ [get_elem(Ms, N, 1)]);
```

Тут перевіряється позиція `(N-1,2)`. Це сусідній до нижнього крайнього елемент побічної діагоналі. У цю позицію записується значення з координат `(N,1)`.

```
row_pro([_X | Xs], I, J, N, Ms, Acc) when I =:= N, J =:= 1 ->
    row_pro(Xs, I, J + 1, N, Ms, Acc ++ [get_elem(Ms, N - 1, 2)]);
```

Тут перевіряється нижній крайній елемент побічної діагоналі, тобто позиція `(N,1)`. У нього записується значення з `(N-1,2)`. Таким чином реалізується друга перестановка.

---

9. Інші елементи матриці

```
row_pro([X | Xs], I, J, N, Ms, Acc) ->
    row_pro(Xs, I, J + 1, N, Ms, Acc ++ [X]);
```

Якщо поточний елемент не належить до жодної з чотирьох спеціальних позицій, він просто переноситься у новий рядок без змін. Це означає, що змінюються тільки крайні елементи побічної діагоналі та їхні сусіди, а решта матриці залишається такою ж.

---

10. Завершення обробки рядка

```
row_pro([], _, _, _, _, Acc) ->
    Acc.
```

Коли всі елементи поточного рядка вже оброблені, функція повертає готовий новий рядок.

---

11. Отримання елемента матриці

```
get_elem(Ms, I, J) ->
    lists:nth(J, lists:nth(I, Ms)).
```

Функція `get_elem/3` повертає елемент матриці за номером рядка `I` і номером стовпця `J`. Спочатку вона бере потрібний рядок, а потім у цьому рядку бере потрібний елемент. Ця функція використовується в тих місцях, де потрібно підставити значення з іншої позиції матриці під час перестановки.

Коротко про алгоритм для захисту

У програмі матриця подана як список списків. Спочатку визначається її розмір. Далі рекурсивно обробляється кожний рядок матриці, а всередині кожного рядка рекурсивно обробляється кожен елемент. Якщо елемент стоїть у позиції, яка відповідає крайньому елементу побічної діагоналі або його сусіду по діагоналі, то виконується заміна значення. В усіх інших випадках елемент копіюється без змін. У результаті формується нова матриця, у якій крайні елементи побічної діагоналі міняються місцями з сусідніми по діагоналі елементами.

Що сказати про рекурсію

У цій роботі використано два рівні рекурсії.
Перша рекурсія реалізована у функції `matrix_pro/5` і проходить по рядках матриці.
Друга рекурсія реалізована у функції `row_pro/6` і проходить по елементах кожного рядка.
Такий підхід відповідає темі лабораторної роботи, оскільки матриця в Erlang розглядається як список списків.

Що показати викладачу в консолі

Після запуску `erl`:

```
c(lab6_var16).
lab6_var16:matrix_print(1).
lab6_var16:matrix_print(2).
```

Короткий текст “що я робив” по пунктах

1. Створив модуль `lab6_var16` та оголосив функції, які будуть доступні для запуску.

2. Задав дві тестові квадратні матриці у функції `matrix_data/1`.

3. Реалізував основну функцію `matrix_main/1`, яка вибирає матрицю, визначає її розмір і запускає обробку.

4. Реалізував функцію `matrix_pro/5`, яка рекурсивно проходить по рядках матриці.

5. Реалізував функцію `row_pro/6`, яка рекурсивно проходить по елементах рядка і виконує перестановку потрібних елементів побічної діагоналі.

6. Реалізував допоміжну функцію `get_elem/3` для доступу до елемента матриці за координатами.

7. Додав функцію `matrix_print/1` для зручного виводу результату по рядках.

8. Скомпілював програму в Erlang shell і перевірив її роботу на двох тестових матрицях.